BRAD DEVICES

Built in Africa, for the World

BradISA V2

Instruction Set Architecture Specification

Revision 1.0 — Kinetic Gen1 — July 2026

Confidential — Brad Devices Proprietary

Contents

1 Introduction **3**
1.1 Why V2 exists . . . 3
1.2 Why 64 opcodes rather than 32 . . . 3
1.3 Why six bits is not new to this project . . . 3
1.4 Data Model . . . 4
1.5 Data Model . . . 4
1.6 Pipeline Models . . . 4

2 Register File **5**
2.1 Why V2 grew the register file . . . 5
2.2 ABI Convention . . . 5
2.3 Why this shape . . . 5
2.4 Decision record . . . 6
2.5 Calling Convention . . . 6

3 Instruction Formats **7**
3.1 RRR — Register-to-Register ALU . . . 7
3.2 RI — Register-Immediate . . . 7
3.3 SI — Store-Immediate . . . 7
3.4 BR — Conditional Branch . . . 8
3.5 IJ — Indirect Jump . . . 8
3.6 JMP — Unconditional Jump and Call . . . 8
3.7 U — Upper Immediate . . . 8
3.8 RET — Return . . . 8
3.9 Encoding diagrams . . . 9
3.10 Two encoding defects in V1, corrected here . . . 9

4 Instruction Set **10**
4.1 Opcode allocation and what it buys . . . 10
4.2 ALU Operations . . . 10
4.3 Memory Operations . . . 10
4.4 Addressing: how V2 reaches memory . . . 10
4.5 Conditional Branches . . . 10
4.6 Call, Return and Indirect Transfer . . . 12
4.7 Atomics and Memory Ordering . . . 12
4.8 Vector escape . . . 12

5 Exception and Interrupt Model **13**
5.1 Machine Status Registers . . . 13
5.2 Status Register Flags . . . 13

6 Performance Characteristics **14**
6.1 IPC Estimates . . . 14
6.2 Dhrystone Estimates . . . 14
6.3 Pipeline Stage Descriptions . . . 14
6.4 Vector Extension (VSET) . . . 14

1 Introduction

BradISA V2 (Kinetic Gen2) is a 32-bit fixed-length RISC instruction set architecture designed for the Brad Devices BradCore processor family. It provides 32 general-purpose registers, 64 opcodes encoded in a 6-bit primary opcode field, and seven instruction formats, each of which is exactly 32 bits wide.

V2 supersedes V1 in full. It is a clean break: every instruction encoding changes, and none of V1's encodings are preserved. That is a deliberate consequence of the field widths — growing a register field from four bits to five shifts every field above it, so no compatibility-preserving arrangement exists once the register file grows. The cost is nil today, because no V1 binary has ever executed on silicon; the same break after a toolchain ships would be a migration.

1.1 Why V2 exists

V1 cannot be compiled. Its RI format carries a 16-bit sign-extended immediate and no instruction loads a wide constant, so the addressable range from any base register is ±32 KiB. There is therefore no data segment beyond 64 KiB, no heap, no array longer than 16,384 elements and no linked executable — which means no C library. V1 has no indirect jump, so no function pointers, no virtual calls and no callbacks.

The three missing instruction groups are a symptom rather than the disease. V1's sixteen opcodes are fully allocated, `0x0 ADD` through `0xF RET`, with none free. The property the V1 design document held up as its central argument — a sixteen-case scalar decoder with no decode hierarchy — is precisely the property that prevents it growing. An ISA cannot have both a small exhaustive opcode space and the ability to address memory.

1.2 Why 64 opcodes rather than 32

An earlier proposal in this review recommended a 5-bit field, 32 opcodes. The arithmetic does not support it. Clang's integer subset alone requires:

Table 1: Integer instructions a compiler emits, against opcode-space size

Group	Instructions	Count
ALU, register-register	ADD SUB MUL MULHU DIV DIVU REM REMU AND OR XOR SHL SHR SAR	12
ALU, register-immediate	ADDI ANDI ORI XORI SLLI SRLI SRAI SLTI SLTIU	9
Load / store	LDW LHU LHBU STW SH SB	6
Upper immediate	LUI AUIPC	2
Control transfer	NOP JMP JAL CALL JALR RET FENCE	6
Branches	BEQ BNE BLT BGE BLTU BGEU BZ BNZ	8
Atomics	LR.W SC.W AMOSWAP.W AMOADD.W	4
Total		50

A 32-slot space is eleven slots short of the fifty instructions above. Closing that gap at 5 bits requires either folding the register-immediate ALU variants behind a `funct` field — reintroducing the sub-hierarchy the V1 design explicitly rejected — or deferring atomics to a third encoding break. V2 takes neither cost and spends one more bit.

1.3 Why six bits is not new to this project

The vector core already implements this encoding.

Table 2: Opcode-space arithmetic

Field width	Slots	Verdict
4 bits	16	V1. Short by 34.
5 bits	32	Short by 18 against 50.
6 bits	64	Fits, with 14 slots reserved and accounted for.

```
src/rtl/bradvector_core.v:27   localparam REG_ADDR_W = 5
src/rtl/bradvector_core.v:28   localparam NUM_REGS     = 32
src/rtl/bradvector_core.v:41   wire [4:0] slot0_opcode
```

The scalar ISA was the outlier that never grew. V2 ports the vector core's proven field widths to the scalar side, so the argument for the change is consistency between two cores already in the same repository rather than a new mechanism.

1.4 Data Model

- **Word size:** 32 bits (4 bytes)
- **Address space:** 32-bit byte-addressed
- **Page size:** 4096 bytes (4 KiB)
- **Endianness:** Little-endian
- **Instruction length:** 32 bits, fixed

1.5 Data Model

- **Word size:** 32 bits (4 bytes)
- **Address space:** 32-bit byte-addressed
- **Page size:** 4096 bytes (4 KiB)
- **Endianness:** Little-endian
- **Instruction length:** 32 bits, fixed

1.6 Pipeline Models

BradISA defines two pipeline models:

Table 3: Pipeline model comparison

Property	Phoenix	Falcon
Pipeline depth	10 stages	5 stages
Issue width	Octa-issue	Single-issue
Execution order	Out-of-order	In-order
ROB size	128 entries	—
Typical IPC	2.2	0.9
Peak IPC	5.0	1.0
Target DMIPS/MHz	5.5	2.2
Target frequency	3.0–5.2 GHz	2.4–3.6 GHz

2 Register File

BradISA V2 provides 32 32-bit general-purpose registers, numbered `r0` through `r31`. Register `r0` is hardwired to zero; writes to `r0` are silently discarded.

2.1 Why V2 grew the register file

V1 provided sixteen registers. Widening the register fields from four bits to five is the same one-bit change as widening the opcode, and it brings the scalar core into line with the vector core, which already ships `REG_ADDR_W = 5` and `NUM_REGS = 32`.

The motivation is not the field width. It is the ABI. V1 allocated its sixteen registers as four arguments, three temporaries, five callee-saved, a stack pointer, a link register and a read-only program counter, which left a register allocator roughly eight usable registers and the calling convention four parameters. Every call past the fourth spilled. A compiler can be written for such an ABI and it will work, but it will spill continuously, and no backend quality survives it.

V2 settles this in §2.2: eight argument registers, six temporaries, eleven callee-saved, and no architectural PC register.

2.2 ABI Convention

Decision. V2 passes arguments in eight registers, holds six temporaries and eleven callee-saved, and **retires the architectural PC register**. The shape is x86-32's argument count on a RISC-V register budget. The PC pseudo-register occupied one of thirty-two slots while being unencodable as `rd`, `rs1` or `rs2`; removing it costs nothing, because `AUIPC` already supplies PC-relative addressing and an explicit "read the PC" operation can take a reserved opcode when one is wanted.

Table 4: Application Binary Interface register map (32 registers)

Register	ABI Name	Count	Purpose
`r0`	`zero`	1	Hardwired zero; writes discarded
`r1`–`r8`	`a0`–`a7`	8	Arguments #0–#7; `a0` also returns
`r9`–`r14`	`t0`–`t5`	6	Caller-saved temporaries
`r15`	`lr`	1	Link register; return address on `CALL`
`r16`–`r26`	`s0`–`s10`	11	Callee-saved
`r27`	`sp`	1	Stack pointer
`r28`	`gp`	1	Global pointer
`r29`	`tp`	1	Thread pointer
`r30`–`r31`	—	2	Reserved

Twenty-nine of thirty-two registers are allocatable: one is hardwired zero and two are reserved for growth. The arithmetic is checked rather than asserted – 8 arguments + 6 temporaries + 11 callee-saved + `lr` + `sp` + `gp` + `tp` = 29.

2.3 Why this shape

V1 allocated sixteen registers as four arguments, three temporaries, five callee-saved, a stack pointer, a link register and a read-only program counter. That left a register allocator roughly eight usable registers and the calling convention four parameters, so every call past the fourth spilled to the stack. A compiler can be written for that ABI and it will work — it will also spill continuously, and no backend quality survives it.

The alternative candidate was RISC-V's seven arguments. Eight was chosen because the register file can afford it, because eight is the better-trodden count for a 32-bit integer ABI, and because every argument beyond the register set costs a store and a reload at both ends of every call.

Sixteen callee-saved and reserved slots remain unallocated on purpose. They are where a vector calling convention, a software-pipeline temporaries group and any future banked or rotate arrangement would go, and they are named as reserved rather than left unassigned.

2.4 Decision record

The encoding was fixed before this convention was chosen, deliberately. Growing a register field from four bits to five shifts every field above it, so the encoding had to be settled first, and the convention is the part that could be decided afterwards on evidence. It cost nothing to leave open for exactly that length of time.

Two open questions became this one. How many arguments — seven or eight — and whether to keep a read-only PC register. Both are answered above.

2.5 Calling Convention

- Arguments 0–7 are passed in `r1`–`r8`. Additional arguments are passed on the stack, right to left.
- Return value is placed in `a0` (`r1`).
- Callee-saved registers `s0`–`s10` must be preserved across function calls. `t0`–`t5` and `a0`–`a7` may be clobbered.
- The link register `lr` holds the return address on `CALL` and `RET`.
- The stack grows downward; `sp` must point to the last used stack slot, and the stack is 16-byte aligned on entry.
- There is no red zone below `sp`, and no leaf-function fast path that omits the frame setup; a signal or exception may overwrite the caller-saved area.

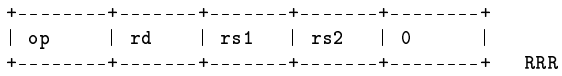
3 Instruction Formats

All BradISA V2 instructions are 32 bits wide. Seven formats are defined, and every one of them sums to exactly 32 bits. That is not a coincidence to be checked by hand later; it is the property the V1 specification failed to hold, see §3.10.

Table 5: Instruction format field layout

Format	[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]
RRR	op	rd	rs1	rs2	—	—
RI	op	rd	rs1	imm[15:0]		
SI	op	rs2	rs1	imm[15:0]		
BR	op	rs1	rs2	offset[15:0]		
IJ	op	rd	rs1	imm[15:0]		
JMP	op	rd	offset[20:0]			
U	op	rd	imm[20:0]			

3.1 RRR — Register-to-Register ALU

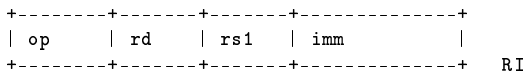


$$\texttt{instruction[31:0]} = \{\, \texttt{op[5:0]}, \texttt{rd[4:0]}, \texttt{rs1[4:0]}, \texttt{rs2[4:0]}, 11'b0 \,\}$$

Bits [10:0] are reserved and must be zero. A non-zero value in a reserved field is an illegal instruction, not a synonym for zero, so that encodings do not acquire two spellings.

3.2 RI — Register-Immediate

Used by ADDI, ANDI, ORI, XORI, the shift-immediates, the set-less-thans and every load.

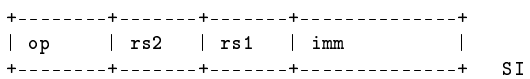


$$\texttt{instruction[31:0]} = \{\, \texttt{op[5:0]}, \texttt{rd[4:0]}, \texttt{rs1[4:0]}, \texttt{imm[15:0]} \,\}$$

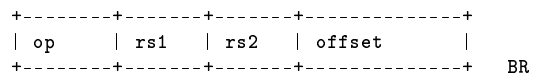
The 16-bit immediate is sign-extended to 32 bits before use, except for `ANDI`, `ORI` and `XORI`, which zero-extend it, because those are bitwise operations and sign extension would be a different operation.

3.3 SI — Store-Immediate

Identical bit layout to RI. The register field at [25:21] is the *source* of the stored value rather than a destination.



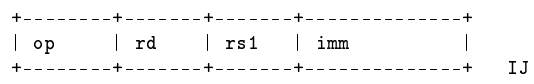
3.4 BR — Conditional Branch



$$\texttt{PC} = \texttt{insn_pc} + 4 + (\texttt{sign_extend(offset[15:0])} \times 4)$$

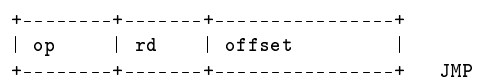
Branch offsets are therefore ±128 KiB from the instruction. `BZ` and `BNZ` use this format with `rs2` = 0, which is why they need no separate encoding.

3.5 IJ — Indirect Jump



`JALR`: the target is computed and the link written before the transfer, so that `jalr ra, ra, 0` is a correct tail call.

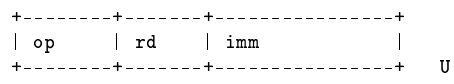
3.6 JMP — Unconditional Jump and Call



$$\texttt{PC} = \texttt{insn_pc} + 4 + (\texttt{sign_extend(offset[20:0])} \times 4)$$

Unconditional jump range is ±4 MiB, unchanged from V1. The V1 encoding spent eight bits on padding; V2 spends five of them on a real `rd`, which is what `JAL` needs to return an address.

3.7 U — Upper Immediate



$$\texttt{LUI}: \quad \texttt{rd} = \texttt{sign_extend(imm[20:0])} \ll 11$$

$$\texttt{AUIPC}: \quad \texttt{rd} = \texttt{PC} + (\texttt{sign_extend(imm[20:0])} \ll 11)$$

21 immediate bits shifted left by 11 fills all 32 bits of the destination, so `LUI` followed by `ADDI` constructs any 32-bit constant. This is the instruction whose absence kept V1 from addressing memory.

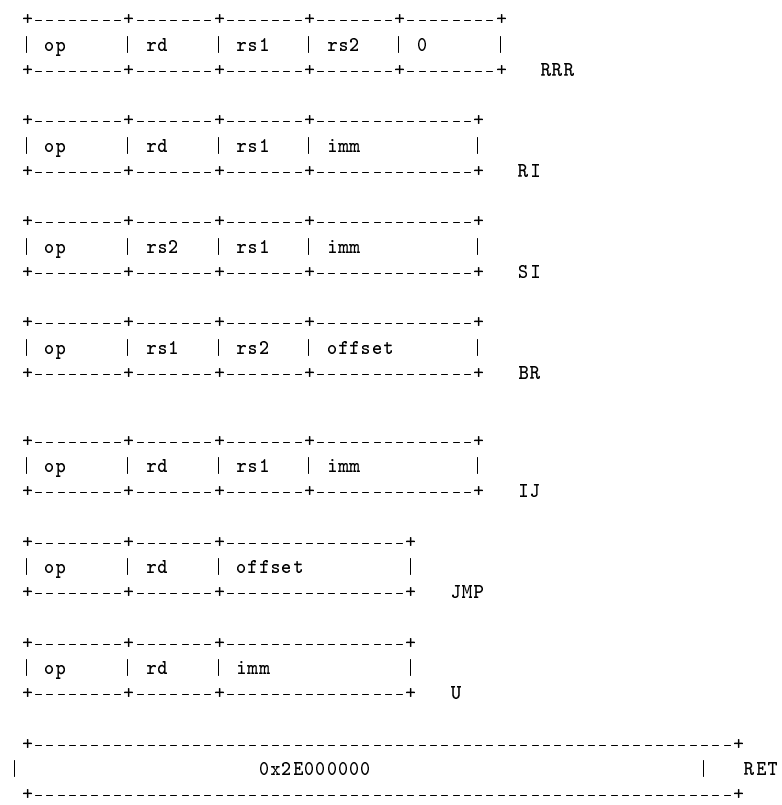
3.8 RET — Return

Fixed encoding, with every non-opcode bit zero:

$$\texttt{RET} = 32\text{'}h\texttt{2E000000}$$

`RET` jumps to the address in `r15 (lr)` and does not modify `lr`.

3.9 Encoding diagrams



3.10 Two encoding defects in V1, corrected here

Recorded because both were load-bearing for anything transcribed from this specification, and because a TableGen target is written directly against this table.

The V1 RI format did not describe a 32-bit word. The field array summed to 28 bits ($4+4+4+16$), four short, and the accompanying diagram placed `imm[15:0]` at bits 31:16, overlapping `op`, `rd` and `rs1`. Two representations of one format, disagreeing, neither of them a 32-bit encoding.

V1 opcode `0xF` carried two meanings. The scalar table assigned `0xF = RET`; the vector extension was specified as using opcode `0xF` for every one of its instructions, disambiguated by a class field in bits [27:25] with `0xF0000000` pinned as the sole `RET` encoding. The scalar and vector decoders therefore disagreed about the same four bits, and the entire vector extension depended on `RET` having exactly one encoding.

V2 gives the vector unit its own opcode range (§4.8) and every format sums to 32 bits.

4 Instruction Set

4.1 Opcode allocation and what it buys

The 64 slots are allocated by group, and every unassigned slot is stated rather than left blank. This is the point of the exercise: an unassigned slot in a flat decoder is not a gap, it is an instruction word that executes as something nobody chose.

4.2 ALU Operations

Register-register ALU instructions use the RRR format. Register-immediate ALU instructions use RI. Shift amounts use only the bottom five bits of the source (0–31), in both forms.

Division and remainder are newly defined in V2 and raise an exception on a zero divisor rather than returning a defined value, because the alternative is a silent wrong answer.

4.3 Memory Operations

`LDW`, `LHU` and `LHBU` load into `rd` from `rs1 + sext(imm)`. `STW`, `SH` and `SB` store the low 32, 16 or 8 bits of `rs2` to the same effective address, and are encoded with the SI format, in which the register field at bits [25:21] is the *source*, not a destination.

V1 used the RI shape for stores and documented this as "STW ignores `rd` (uses `rs2` as store data)". That is the same bit layout read with two different meanings depending on the opcode, which is exactly the sort of thing that costs a backend an afternoon. V2 names the two cases separately.

Word accesses must be 4-byte aligned and halfword accesses 2-byte aligned. An unaligned access raises an exception; it is not silently split.

4.4 Addressing: how V2 reaches memory

This is the change that makes a compiler possible.

In V1 the only addressable range from any base register was ±32 KiB, because the RI immediate was 16 bits and no instruction loaded a wide constant. V2 adds the U format:

```
lui   r5, 0x123       # r5 = 0x123000
addi  r5, r5, 0x456   # r5 = 0x123456
```

Any 32-bit address is therefore two instructions, and any load or store reaches ±32 KiB around an arbitrary base. `AUIPC` supplies the PC-relative form for position-independent code.

4.5 Conditional Branches

The BR format carries two registers, so compare-and-branch is a single instruction. V1 had only `BZ` and `BNZ`, which test one register against zero; every other comparison compiled to a `SUB` followed by a zero test.

Branches are PC-relative against the address of the instruction itself:

$$\texttt{target} = \texttt{insn_pc} + 4 + (\texttt{sign_extend(offset)} \times 4)$$

The offset is a 16-bit signed word offset, giving ±128 KiB of range. That is a deliberate choice: with AUIPC available, a branch further than 128 KiB is two instructions, and spending bits on branch range would have cost either the immediate width or the opcode width, both of which are load-bearing.

Table 6: Complete BradISA V2 integer instruction set

Opcode	Mnemonic	Format	Operation
0x0	NOP	—	no operation
0x1	ADD	RRR	rd = rs1 + rs2
0x2	SUB	RRR	rd = rs1 - rs2
0x3	MUL	RRR	rd = rs1 × rs2 (low 32 bits)
0x4	MULHU	RRR	rd = (rs1 × rs2)[63:32] unsigned
0x5	DIV	RRR	rd = rs1 / rs2 signed, div-by-zero raises an exception
0x6	DIVU	RRR	rd = rs1 / rs2 unsigned
0x7	REM	RRR	rd = rs1 % rs2 signed
0x8	REMU	RRR	rd = rs1 % rs2 unsigned
0x9	AND	RRR	rd = rs1 & rs2
0xA	OR	RRR	rd = rs1 | rs2
0xB	XOR	RRR	rd = rs1 ^ rs2
0xC	SHL	RRR	rd = rs1 << rs2[4:0]
0xD	SHR	RRR	rd = rs1 >> rs2[4:0] (logical)
0xE	SAR	RRR	rd = rs1 $\gg_s$ rs2[4:0] (arithmetic)
0xF	—	—	reserved
0x10	ADDI	RI	rd = rs1 + sext(imm)
0x11	ANDI	RI	rd = rs1 & zext(imm)
0x12	ORI	RI	rd = rs1 | zext(imm)
0x13	XORI	RI	rd = rs1 ^ zext(imm)
0x14	SLLI	RI	rd = rs1 << imm[4:0]
0x15	SRLI	RI	rd = rs1 >> imm[4:0] (logical)
0x16	SRAI	RI	rd = rs1 $\gg_s$ imm[4:0] (arithmetic)
0x17	SLTI	RI	rd = (rs1 < sext(imm)) ? 1 : 0
0x18	SLTIU	RI	rd = (rs1 < zext(imm)) ? 1 : 0
0x19–0x1B	—	—	reserved
0x1C–0x1F	—	—	reserved
0x20	LDW	RI	rd = mem[rs1 + sext(imm)]
0x21	LHU	RI	rd = zeroextend(mem16[rs1 + sext(imm)])
0x22	LHBU	RI	rd = zeroextend(mem8[rs1 + sext(imm)])
0x23	STW	SI	mem[rs1 + sext(imm)] = rs2
0x24	SH	SI	mem16[rs1 + sext(imm)] = rs2[15:0]
0x25	SB	SI	mem8[rs1 + sext(imm)] = rs2[7:0]
0x26–0x27	—	—	reserved
0x28	LUI	U	rd = sext(imm) << 11
0x29	AUIPC	U	rd = PC + (sext(imm) << 11)
0x2A	JMP	JMP	PC = insn_pc + 4 + sext(offset × 4)
0x2B	JAL	JMP	rd = insn_pc + 4; PC = target
0x2C	CALL	JMP	lr = insn_pc + 4; PC = target
0x2D	JALR	IJ	t = insn_pc + 4; PC = (rs1 + sext(imm)) & ∼1; rd = t
0x2E	RET	—	PC = lr
0x2F	FENCE	—	ordering barrier; no operand
0x30	BEQ	BR	if (rs1 == rs2) branch
0x31	BNE	BR	if (rs1 != rs2) branch
0x32	BLT	BR	if (rs1 < rs2, signed) branch
0x33	BGE	BR	if (rs1 ≥ rs2, signed) branch
0x34	BLTU	BR	if (rs1 < rs2, unsigned) branch
0x35	BGEU	BR	if (rs1 ≥ rs2, unsigned) branch
0x36	BZ	BR	if (rs1 == 0) branch (rs2 = 0)
0x37	BNZ	BR	if (rs1 != 0) branch (rs2 = 0)

Table 7: Opcode group map

Range	Group	Slots	Spare
0x00–0x0F	ALU, register-register	15	1
0x10–0x1F	ALU, register-immediate	9	7
0x20–0x27	Load / store	6	2
0x28–0x29	Upper immediate	2	0
0x2A–0x2F	Control transfer	6	0
0x30–0x37	Branches	8	0
0x38–0x3B	Atomics	4	0
0x3C–0x3F	Extension escape	0	4
Total		**50**	**14**

4.6 Call, Return and Indirect Transfer

`CALL` writes the return address into the link register and jumps. `RET` jumps to the link register. `JALR` is the indirect form, and is the instruction that makes function pointers, virtual calls and callbacks expressible — V1 had no encoding for them at all, so plain C could not be compiled against the ISA.

`JALR` computes the target before writing the link, so a call through `lr` behaves correctly:

```
jalr  ra, ra, 0       # t = pc+4; pc = ra + 0; ra = t  -> correct tail position
```

4.7 Atomics and Memory Ordering

`LR.W` and `SC.W` provide a load-reserved / store-conditional pair; `AMOSWAP.W` and `AMOADD.W` provide unconditional atomic memory operations. `FENCE` is an ordering barrier with no operands.

The V2 atomic set is deliberately minimal — four instructions, which is what a working single-threaded and coarse-grained lock implementation needs. Bitwise atomic operations (AND, OR, XOR) are not in V2. They are not forgotten: the seven spare register-immediate slots and the four extension slots are where they go when codegen demand is measured rather than guessed.

4.8 Vector escape

Slots 0x3C–0x3F are reserved for extensions. The vector unit occupies this space with a class field in the bits below the opcode, so vector instructions are decoded from the same 64-entry table rather than sharing an opcode with a scalar instruction.

In V1 the entire vector extension shared opcode `0xF` with `RET`, and was distinguished only by the value of the remaining bits, with `0xF0000000` pinned as the sole `RET` encoding. That is coherent as written and fragile in practice: the whole vector extension depends on `RET` having exactly one encoding, and any future 32-bit pattern that collides becomes a vector instruction silently. V1 also shared opcode `0xF` with the vector opcode, so the scalar and vector decoders disagreed about what the same four bits meant. V2 removes both by giving the vector unit its own opcode range.

5 Exception and Interrupt Model

BradISA V2 supports a vectored exception model with a fixed vector table at the base of the address space.

Table 8: Exception vector addresses

Vector address	Exception	Cause code
0x00000000	Reset / cold boot	—
0x00000004	Undefined instruction	1
0x00000008	Page fault	2
0x0000000C	Unaligned access	3
0x00000010	Privilege violation	4
0x00000014	System call (ECALL)	5
0x00000018	Timer interrupt	6
0x00000020	External IRQ 0	7
0x00000024	External IRQ 1	7
0x00000028	External IRQ 2	7
0x0000002C	External IRQ 3	7

5.1 Machine Status Registers

Table 9: MSR index map

Index	Mnemonic	Description
0	STATUS	Processor status flags
1	CAUSE	Cause of last exception
2	EPC	Exception program counter
3	EAR	Exception address register
4	PAGE_BASE	Page table base address
5	TICK	Cycle count timer
6	CORE_ID	Core identifier
7	CLUSTER_ID	Cluster identifier

5.2 Status Register Flags

Bit	Flag	Description
0	IE	Interrupt enable
1	SVC	Supervisor mode
2	HALTED	Core halted
3	WFE	Wait for event
8	EXC_PEND	Exception pending

6 Performance Characteristics

6.1 IPC Estimates

- **Phoenix (10-stage OoO):** 2.2 IPC typical, 5.0 IPC peak
- **Falcon (5-stage in-order):** 0.9 IPC typical, 1.0 IPC peak

6.2 Dhrystone Estimates

- **Phoenix:** ∼5.5 DMIPS/MHz
- At 5.2 GHz: ∼28,600 DMIPS
- **Falcon:** ∼2.2 DMIPS/MHz
- At 3.6 GHz: ∼7,920 DMIPS

6.3 Pipeline Stage Descriptions

Phoenix (10-stage)

1. **Fetch1** — Instruction TLB access, next-PC selection
2. **Fetch2** — Instruction cache read
3. **Decode** — Instruction decode, register rename
4. **Rename** — Register map read, free-list access
5. **Dispatch** — Insert into issue queue / ROB
6. **Issue** — Wakeup, select, read physical register file
7. **Execute** — ALU / AGU / branch resolution
8. **Memory** — Data cache access (load/store)
9. **Writeback** — Result write to physical register file
10. **Commit** — Retire from ROB, update architectural state

Falcon (5-stage)

1. **Fetch1** — Instruction fetch, PC increment
2. **Fetch2** — Instruction cache / memory read
3. **Decode** — Instruction decode, register file read
4. **Execute** — ALU / AGU / branch resolution, writeback
5. **Writeback** — Register file write

6.4 Vector Extension (VSET)

BradISA V2 has no floating-point or SIMD facility; both are provided by the BradISA V2 Vector Extension (VSET), specified in the companion document **BradISA V2 Extensions**. VSET adds 32 × 256-bit vector registers (`v0`–`v31`), elementwise integer/FP operations, vector memory, reductions, and converts, encoded under primary opcode `0xF` with `0xF0000000` retained as `RET` for backward compatibility. Enable via MSR 23 (VSTATUS.VE).

Revision History

Revision	Date	Changes
1.0	July 2026	Initial specification — Kinetic Gen1
1.1	July 2026	Phoenix upgraded to octa-issue OoO (ROB 128), IPC 2.2/5.0, DMIPS/MHz 5.5; Falcon DMIPS/MHz aligned to 2.2
1.2	July 2026	Cross-reference BradISA V2 Vector Extension (VSET), opcode `0xF`, VSTATUS MSR 23